Лекция 4. Язык Event-B

Цель лекции

Изучить язык Event-B и среду моделирования Rodin.

Содержание

1. Введение
2. Синтаксис
3. Типы данных и выражения
4. Семантика
5. Уточнение машины

Почему Event-B

- Сертификация по 4 уровню доверия к СЗИ требует разработать формальную модель управления доступом и верифицировать ее с применением инструментальных средств
- ГОСТ Р 59453.X-2021 Защита информации. Формальная модель управления доступом
- Event-B/Rodin - основные средства моделирования и верификации формальных моделей управления доступом

Свойства Event-B/Rodin

- формальный
- статически типизированный
- минималистичность
- математическая основа — теория множеств
- наличие инструментальных средств для написания моделей на Event-B и верификации (Rodin), анимации (ProB) и др.

Дискретно-событийное моделирование

- система обладает состоянием
- событие — воздействие на систему, система мгновенно дает отклик и может изменить свое состояние
- события упорядочены
- система может включать программы, аппаратуру, внешнюю среду

Моделирование — это не программирование

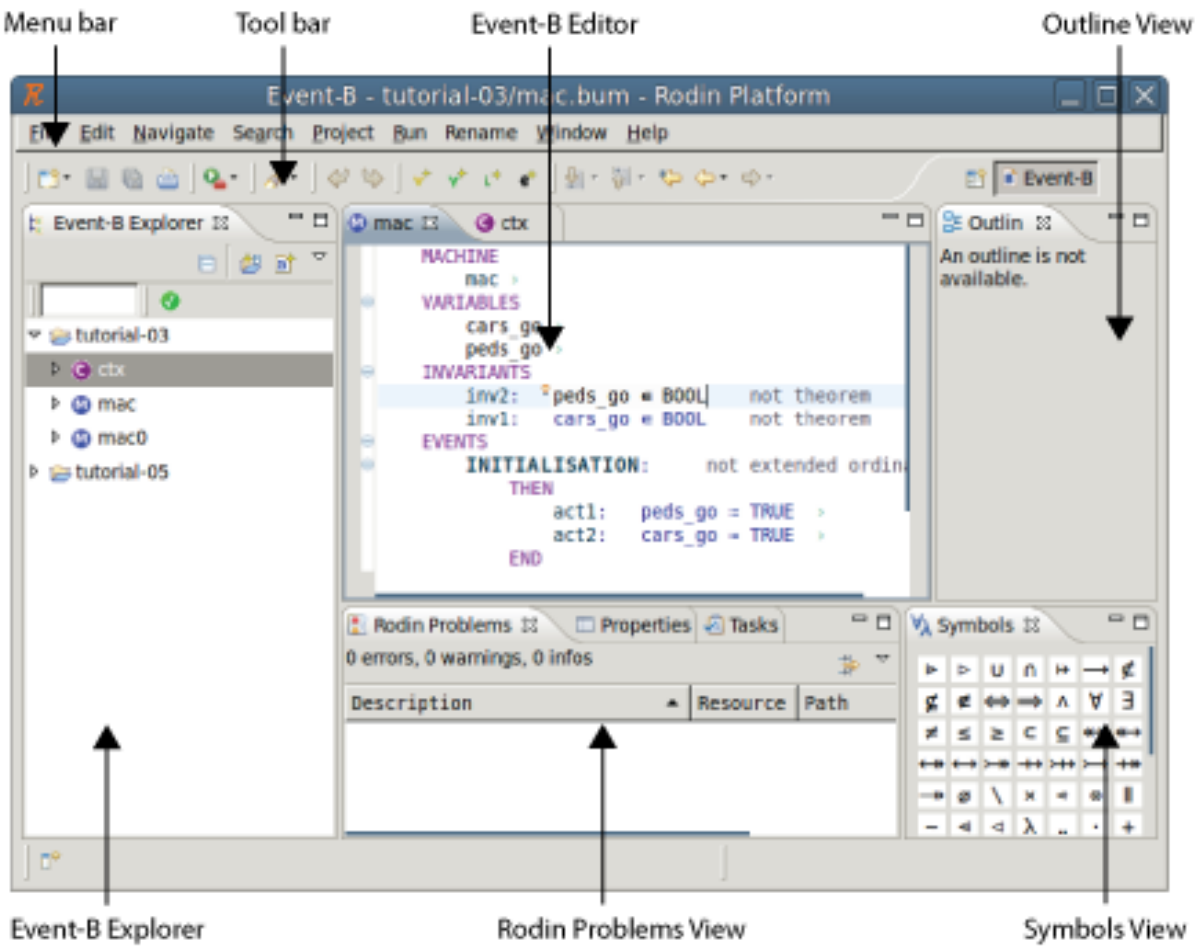
Цель этой деятельности — не написание алгоритма/программы на "еще одном языке а получение правильных требований к программе/системе:

1. Точные, недвусмысленные формулировки
2. Абстракция (отсутствие лишних деталей $\rightarrow$ проще)
3. Доказана непротиворечивость
4. Возможна поэтапная разработка модели (добавление информации небольшими порциями с постоянным доказательством корректности)

Основные элементы формального метода Event-B

- статические свойства системы — ‘context‘
- динамические свойства системы — ‘machine‘
- формальная модель — множество контекстов и последовательность машин
- контест может «расширять» другие контексты
- машина может «видеть» несколько контекстов
- машина может «уточнять» другую машину

Как это выглядит в Rodin



Содержание

1 Введение

2 Синтаксис

3 Типы данных и выражения

4 Семантика

5 Уточнение машины

Контекст

- название контекста
- расширяемые контексты-предки
- несущие множества
- константы
- аксиомы/теоремы

Машина

- название машины
- уточняемая абстрактная машина
- контексты
- переменные
- инварианты
- события (в т.ч. INITIALISATION)

Событие

- название события
- уточняемое абстрактное событие
- параметры
- охранные условия
- действия

Содержание

1 Введение

2 Синтаксис

3 Типы данных и выражения

4 Семантика

5 Уточнение машины

Простейшие типы данных

- `INT`
- `NAT`
- `NAT1`
- `BOOL`
- несущие множества

Выражения — общие положения

- статически типизированы
- могут быть неопределены (пример: делят на 0), есть wd-условие
- логика первого порядка: предикаты и выражения синтаксически различаются (значением переменной не может быть предикат, но может быть функция)

Логические выражения

- константы `TRUE`, `FALSE`
- логические связки `&`, `or`, `=>`, `<=>`, `not`
- логика короткая, при использовании различных связок в одном выражении скобки необходимы
- квантор всеобщности `!`
- квантор существования `#`
- сравнения `=`, `/=`, `>`, ...
- принадлежность `:`, `/:`, `<<`, ...

Выражения с множествами

- пустое множество `{}`
- перечисление `{1, 2, 3}`
- генератор `{x * 2 | x : 1 .. 10}`
- целочисленные диапазоны (обе границы включаются) `a .. b`
- теоретико-множественные операции \/, /\, \

Предопределенные функции и предикаты

- предикат как выражение типа BOOL `bool(predicate)`
- конечность множества `finite(set)`
- размер множества `card(set)`
- равенство объединению непересекающихся множеств `partition(set, set1, set2, ..., setN)`

Пары и отношения

- пара `a |-> b`
- декартово произведение `A ** B`
- множество подмножеств `POW(A)`, `POW1(A)`
- произвольное отношение `A <-> B`
- тотальное отношение `A <<-> B`
- сюръективное отношение `A <->> B`
- тотальное сюръективное отношение `A <<->> B`

Операции над отношениями

- область определения (домен) `dom(relation)`
- область значений `ran(relation)`
- проекция `relation[set]`
- оставить только поддомен `set <| relation`
- убрать поддомен `set <<| relation`
- оставить только значения `relation |> set`
- убрать значения `relation |>> set`

Функции

Функция — это детерминированное подмножество отношения.

- множество тотальных функций `A -> B`
- множество частичных функций `A +-> B`
- применение `function(arg)`

Действия

- одиночное `x := E`
- множественное `x1, x2, ..., xN := E1, E2, ..., EN`
- замена пары в отношении `x(F) := E`
- неявное одиночное
  `x :| before-after-predicate with x and x'`
- неявное множественное
  `x1, ..., xN :| b-a-p with x1, ..., xN, x1', ..., xN'`
- неявный выбор из непустого множества `x :: set`

Содержание

1 Введение

2 Синтаксис

3 Типы данных и выражения

4 Семантика

5 Уточнение машины

Семантика контекстов

1. Несущие множества и константы — символы
2. Интерпретация символов — это сопоставление им математических значений, при котором все аксиомы истинны
3. Формула является следствием из аксиом, если она истинна для всех интерпретаций
4. К условию верификации нужно относиться как к следствию из аксиом

Функционирование машины

1. состояние машины — значения всех переменных
2. событие — переход из одного состояния в другое
3. событие может иметь параметры
4. переход возможен, только если истинны все охранные условия
5. в этом случае выполняются все действия
6. действия одного события выполняются одновременно
7. начальное состояние — результат события `INITIALISATION`

Корректно определенная машина/контекст

1. `WD`: нет неопределенных выражений
2. `INV`: не нарушены инварианты
3. `THM`: не нарушены теоремы
4. `FIS`: неявные действия разрешимы
5. `GRD`, `SIM`, `EQL`: уточнение корректно

Rodin генерирует условия верификации для формального доказательства, что машина корректно определена.

Содержание

1 Введение

2 Синтаксис

3 Типы данных и выражения

4 Семантика

5 Уточнение машины

Цель уточнения

- Уточнение — это отношение между двумя машинами, абстрактной и конкретной, при котором все свойства абстрактной машины справедливы для конкретной.
- Вертикальное уточнение (уточнение данных) — конкретная машина понижает уровень абстракции (например, заменяет абстрактные значения на значения, близкие к реализации)
- Горизонтальное уточнение (уточнение суперпозицией) — конкретная машина оставляет уровень абстракции тем же, но добавляет новые элементы машины (переменные, события).

Состояния машин

- В общем случае, состояния машин не связаны и могут отличаться (вертикальное уточнение)
- Если конкретная переменная использует то же имя переменной, что есть в абстрактной машине, у этих переменных одинаковое значение
- Склеивающий инвариант — инвариант, описывающий соответствие между состояниями двух машин: состоянию конкретной машины соответствуют все состояния абстрактной машины, при которых склеивающий инвариант истинен

События машин

- Каждое событие конкретной машины должно указать одно из событий абстрактной машины (если не указано, то используется skip, то есть пустое действие)
- Параметры события конкретной машины могут отличаться от параметров абстрактной машины
- Для параметров абстрактной машины, отсутствующих в конкретной машине, нужно описать witness-условие в событии конкретной машины
- Witness-условие — это предикат от параметров события конкретной машины и параметра абстрактной машины, описывающий соответствие между ними
- Имя witness-условия должно совпадать с именем параметра абстрактной машины

Трассовое понимание уточнения

- Конкретная трасса — это цепочка состояний конкретной машины, между которыми находятся переходы конкретной машины такие, что все охранные условия и действия выполнены
- Абстрактная трасса — аналогично для абстрактной машины
- Каждой конкретной трассе должна соответствовать хотя бы одна абстрактная трасса такая, что:
  1. Каждому состоянию в конкретной трассе соответствует некоторое состояние в абстрактной трассе такое, что выполнен *склеивающий инвариант*
  2. Каждому переходу в конкретной трассе соответствует некоторый переход в абстрактной машине

Трассовое понимание уточнения (2)

Иными словами, абстрактная машина описывает ограничения на безопасные трассы, а трассам конкретной машины должны соответствовать только безопасные трассы абстрактной машины.

Условия верификации

Rodin генерирует условия верификации для формального доказательства этого свойства:

1. если охранное условие конкретного события истинно, то охранное условие абстрактного события тоже истинно
2. для каждого конкретного перехода $C_1 \xrightarrow{e_C} C_2$ и каждого абстрактного перехода $A_1 \xrightarrow{e_A} A_2$ для соответствующего события каждому A_1, склеенному с C_1, и A_2, склеенному с C_2, A_2 должен получаться в результате действия события e_A из состояния A_1

Синтаксис уточнения

- `machine M2 refines M1 ... end` задает, какая машина является абстрактной (`M1`), а какая — конкретной (`M2`)
- `event e2 refines e1 ... end` задает, на какое абстрактное событие (`e1`) указывает конкретное событие (`e2`)
- `event ... any a b c with @p: predicate(p, a, b, c) ...` задает witness-условие для абстрактного параметра p, отсутствующего среди конкретных параметров (a, b, c)

События-расширения

Горизонтальное уточнение задается при помощи *расширения событий*:

1. параметры только добавляются
2. охранные условия только добавляются
3. действия только добавляются

В Rodin у таких событий написано `extended` и редактор показывает, но не позволяет редактировать, части абстрактного события